OpenMV-MicroPython-Bibliotheken

csi, ml, os, gc – Computer Vision mit KI auf dem Mikrocontroller

schülerforschungszentrum südwürttemberg

1. csi – Kamerasensoren

```python
# Anwendungsbeispiel
import csi
# Kamera einrichten
csi0 = csi.CSI()            # Objekt der Klasse CSI erstellen
csi0.reset()                # Sensor rücksetzen und initialisieren
csi0.pixformat(csi.RGB565)  # Pixelformat RGB565 (5-Bit Rot, 6-Bit Grün, 5-Bit Blau)
csi0.framesize(csi.QVGA)    # Framesize QVGA (320x240)
csi0.window((240, 240))     # Auflösung der Kamera auf Teilbereich setzen
csi0.vflip(True)            # vertikaler Kippmodus
csi0.hmirror(True)          # horizontaler Spiegelmodus
csi0.snapshot(time=2000)    # blockieren, damit die Einstellungen wirksam werden
# Fotos machen
while True:
    img = csi0.snapshot()   # Foto machen und image.Image Objekt zurückgeben
```

Das **sensor**-Modul ist veraltet. Verwende stattdessen das neue **csi**-Modul. Dem sensor-Modul werden keine neuen Funktionen hinzugefügt, und es kann in einer zukünftigen Version entfernt werden.

2. ml – Maschinelles Lernen

Das Modul **ml** enthält Funktionen zum Laden und Ausführen von TensorFlow-Lite-Modellen auf der OpenMV Cam.

class ml.Model(path: str, *, postprocess: object = None) → Model
Lädt ein TensorFlow-Lite-Modell aus path in den Speicher und bereitet es für die Inferenz vor. path ist eine Datei im Dateisystem (oder der Name eines Modells in der Firmware). postprocess ist ein optionales aufrufbares Objekt zur Nachbearbeitung, das von Model.predict nach der Inferenz aufgerufen wird.

predict(inputs: list, *, callback: object = None) → list
Führt die Inferenz für das Modell aus und gibt die Ausgabetensoren zurück.
inputs ist eine Liste mit einem Eintrag pro Eingabetensor des Modells. Jeder Eintrag kann ein ein image.Image Objekt sein (oder ein ndarray, oder ein aufrufbares Objekt).

3. os - grundlegende „Betriebssystem“-Dienste

Das Modul **os** enthält Funktionen für den Dateisystemzugriff und das Einbinden, für Terminal-Umleitung und -Duplizierung sowie die Funktionen uname und urandom.

os.stat(path: str) → Tuple
Ruft den Status einer Datei oder eines Verzeichnisses ab. Dateigröße: os.stat(path: str)[6]

Das **u**-prefix wurde von den MicroPython Modulen entfernt.

4. gc - Steuerung des Garbage Collectors

gc.mem_free() → int
Gibt die Anzahl der Bytes an Heap-RAM zurück, die zur Allokation durch Python-Code verfügbar sind, oder -1, wenn diese Menge nicht bekannt ist.

Quelle: https://docs.openmv.io/dev/de/library/index.html